Relational Database Model

Functional Dependency:

A functional dependency is a constraint that specifies the relationship between two sets of attributes where one set can accurately determine the value of other sets. It is denoted as **X** → **Y**, where X is a set of attributes that is capable of determining the value of Y. The attribute set on the left side of the arrow, **X** is called **Determinant**, while on the right side, **Y** is called the **Dependent**. Functional dependencies are used to mathematically express relations among database entities and are very important to understand advanced concepts in Relational Database System and understanding problems in competitive exams like Gate.

Example:

roll_no	name	dept_name	dept_building
42	abc	CO	A4
43	pqr	IT	A3
44	xyz	CO	A4
45	xyz	IT	A3
46	mno	EC	B2
47	jkl	ME	B2

From the above table we can conclude some valid functional dependencies:

- roll_no → { name, dept_name, dept_building },→ Here, roll_no can determine values of fields name, dept_name and dept_building, hence a valid Functional dependency

- roll_no → dept_name , Since, roll_no can determine whole set of {name, dept_name, dept_building}, it can determine its subset dept_name also.
- dept_name → dept_building , Dept_name can identify the dept_building accurately, since departments with different dept_name will also have a different dept_building
- More valid functional dependencies: roll_no → name, {roll_no, name} ⇢ {dept_name, dept_building}, etc.

Here are some invalid functional dependencies:

- name → dept_name Students with the same name can have different dept_name, hence this is not a valid functional dependency.
- dept_building → dept_name There can be multiple departments in the same building, For example, in the above table departments ME and EC are in the same building B2, hence dept_building → dept_name is an invalid functional dependency.
- More invalid functional dependencies: name → roll_no, {name, dept_name} → roll_no, dept_building → roll_no, etc.

Armstrong's axioms/properties of functional dependencies:

1. **Reflexivity:** If Y is a subset of X, then X→Y holds by reflexivity rule
   For example, {roll_no, name} → name is valid.
2. **Augmentation:** If X → Y is a valid dependency, then XZ → YZ is also valid by the augmentation rule.
   For example, If {roll_no, name} → dept_building is valid, hence {roll_no, name, dept_name} → {dept_building, dept_name} is also valid.→
3. **Transitivity**: If X → Y and Y → Z are both valid dependencies, then X→Z is also valid by the Transitivity rule.
   For example, roll_no → dept_name & dept_name → dept_building, then roll_no → dept_building is also valid.

Types of Functional dependencies in DBMS:

1. Trivial functional dependency
2. Non-Trivial functional dependency
3. Multivalued functional dependency
4. Transitive functional dependency

1. Trivial Functional Dependency

In **Trivial Functional Dependency**, a dependent is always a subset of the determinant.
i.e. If **X → Y** and **Y is the subset of X**, then it is called trivial functional dependency

For example,

roll_no	name	age

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Here, **{roll_no, name}** → **name** is a trivial functional dependency, since the dependent **name** is a subset of determinant set **{roll_no, name}**
Similarly, **roll_no** → **roll_no** is also an example of trivial functional dependency.

2. Non-trivial Functional Dependency

In **Non-trivial functional dependency**, the dependent is strictly not a subset of the determinant.
i.e. If **X** → **Y** and **Y is not a subset of X**, then it is called Non-trivial functional dependency.
For example,

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Here, **roll_no** → **name** is a non-trivial functional dependency, since the dependent **name** is **not a subset of** determinant **roll_no**
Similarly, **{roll_no, name}** → **age** is also a non-trivial functional dependency, since **age** is **not a subset of {roll_no, name}**

3. Multivalued Functional Dependency

In **Multivalued functional dependency**, entities of the dependent set are **not dependent on each other.**

i.e. If **a → {b, c}** and there exists **no functional dependency** between **b and c**, then it is called a **multivalued functional dependency.**

For example,

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18
45	abc	19

Here, **roll_no → {name, age}** is a multivalued functional dependency, since the dependents **name** & **age** are **not dependent** on each other(i.e. **name → age** or **age → name doesn't exist !**)

4. Transitive Functional Dependency

In transitive functional dependency, dependent is indirectly dependent on determinant.

i.e. If **a → b** & **b → c**, then according to axiom of transitivity, **a → c**. This is a **transitive functional dependency**

For example,

enrol_no	name	dept	building_no
42	abc	CO	4
43	pqr	EC	2
44	xyz	IT	1

enrol_no	name	dept	building_no
45	abc	EC	2

Here, **enrol_no** → **dept** and **dept** → **building_no**,
Hence, according to the axiom of transitivity, **enrol_no** → **building_no** is a valid functional dependency. This is an indirect functional dependency, hence called Transitive functional dependency.

Partial Dependency and Fully Functional Dependency

Fully Functional Dependency :
If X and Y are an attribute set of a relation, Y is fully functional dependent on X, if Y is functionally dependent on X but not on any proper subset of X.
Example –
In the relation ABC->D, attribute D is fully functionally dependent on ABC and not on any proper subset of ABC. That means that subsets of ABC like AB, BC, A, B, etc cannot determine D.
Let us take another example –
Supply table

supplier_id	item_id	price
1	1	540
2	1	545
1	2	200

2	2	201
1	1	540
2	2	201
3	1	542

From the table, we can clearly see that neither supplier_id nor item_id can uniquely determine the price but both supplier_id and item_id together can do so. So we can say that price is fully functionally dependent on { supplier_id, item_id }. This summarizes and gives our fully functional dependency −

```
{ supplier_id , item_id } -> price
```

Partial Functional Dependency:
A functional dependency X->Y is a partial dependency if Y is functionally dependent on X and Y can be determined by any proper subset of X.
For example,
we have a relationship AC->B, A->D, and D->B.
Now if we compute the closure of $\{A^+\}$=ADB
Here A is alone capable of determining B, which means B is partially dependent on AC.
Let us take another example –
Student table

name	**roll_no**	**course**
Ravi	2	DBMS
Tim	3	OS

John	5	Java

Here, we can see that both the attributes name and roll_no alone are able to uniquely identify a course. Hence we can say that the relationship is partially dependent.

Differences between Full Functional Dependency and Partial Functional Dependency:

	Full Functional Dependency	Partial Functional Dependency
1.	A functional dependency X->Y is a fully functional dependency if Y is functionally dependent on X and Y is not functionally dependent on any proper subset of X.	A functional dependency X->Y is a partial dependency if Y is functionally dependent on X and Y can be determined by any proper subset of X.
2.	In full functional dependency, the non-prime attribute is functionally dependent on the candidate key.	In partial functional dependency, the non-prime attribute is functionally dependent on part of a candidate key.
3.	In fully functional dependency, if we remove any attribute of X, then the dependency will not exist anymore.	In partial functional dependency, if we remove any attribute of X, then the dependency will still exist.
4.	Full Functional Dependency equates to the normalization standard of Second Normal Form.	Partial Functional Dependency does not equate to the normalization standard of Second Normal Form. Rather, 2NF eliminates the Partial Dependency.
5.	An attribute A is fully functional dependent on another attribute B if it is functionally dependent on that attribute, and not on	An attribute A is partially functional dependent on other attribute B if it is functionally dependent on any part

	Full Functional Dependency	Partial Functional Dependency
	any part (subset) of it.	(subset) of that attribute.
6.	Functional dependency enhances the quality of the data in our database.	Partial dependency does not enhance the data quality. It must be eliminated in order to normalize in the second normal form.

Inference Rule (IR):

- The Armstrong's axioms are the basic inference rule.
- Armstrong's axioms are used to conclude functional dependencies on a relational database.
- The inference rule is a type of assertion. It can apply to a set of FD(functional dependency) to derive other FD.
- Using the inference rule, we can derive additional functional dependency from the initial set.

The Functional dependency has 6 types of inference rule:

1. Reflexive Rule (IR_1)

In the reflexive rule, if Y is a subset of X, then X determines Y.

If $X \supseteq Y$ then $X \rightarrow Y$

Example:

1. X = {a, b, c, d, e}
2. Y = {a, b, c}

2. Augmentation Rule (IR_2)

The augmentation is also called as a partial dependency. In augmentation, if X determines Y, then XZ determines YZ for any Z.

1. If X → Y then XZ → YZ

Example: For R(ABCD), **if** A → B then AC → BC

3. Transitive Rule (IR_3)

In the transitive rule, if X determines Y and Y determine Z, then X must also determine Z.

If X → Y and Y → Z then X → Z

4. Union Rule (IR_4)

Union rule says, if X determines Y and X determines Z, then X must also determine Y and Z.

If X → Y and X → Z then X → YZ

Proof:

1. X → Y (given)
2. X → Z (given)
3. X → XY (using IR_2 on 1 by augmentation with X. Where XX = X)
4. XY → YZ (using IR_2 on 2 by augmentation with Y)
5. X → YZ (using IR_3 on 3 and 4)

5. Decomposition Rule (IR_5)

Decomposition rule is also known as project rule. It is the reverse of union rule.

This Rule says, if X determines Y and Z, then X determines Y and X determines Z separately.

If X → YZ then X → Y and X → Z

Proof:

1. X → YZ (given)
2. YZ → Y (using IR_1 Rule)
3. X → Y (using IR_3 on 1 and 2)

6. Pseudo transitive Rule (IR_6)

In Pseudo transitive Rule, if X determines Y and YZ determines W, then XZ determines W.

If X → Y and YZ → W then XZ → W

Proof:

1. X → Y (given)
2. WY → Z (given)
3. WX → WY (using IR_2 on 1 by augmenting with W)
4. WX → Z (using IR_3 on 3 and 2)

Normalization

A large database defined as a single relation may result in data duplication. This repetition of data may result in:

- Making relations very large.
- It isn't easy to maintain and update data as it would involve searching many records in relation.
- Wastage and poor utilization of disk space and resources.
- The likelihood of errors and inconsistencies increases.

So to handle these problems, we should analyze and decompose the relations with redundant data into smaller, simpler, and well-structured relations that are satisfy desirable properties. Normalization is a process of decomposing the relations into relations with fewer attributes.

What is Normalization?

- Normalization is the process of organizing the data in the database.

- Normalization is used to minimize the redundancy from a relation or set of relations. It is also used to eliminate undesirable characteristics like Insertion, Update, and Deletion Anomalies.
- Normalization divides the larger table into smaller and links them using relationships.
- The normal form is used to reduce redundancy from the database table.

Why do we need Normalization?

The main reason for normalizing the relations is removing these anomalies. Failure to eliminate anomalies leads to data redundancy and can cause data integrity and other problems as the database grows. Normalization consists of a series of guidelines that helps to guide you in creating a good database structure.

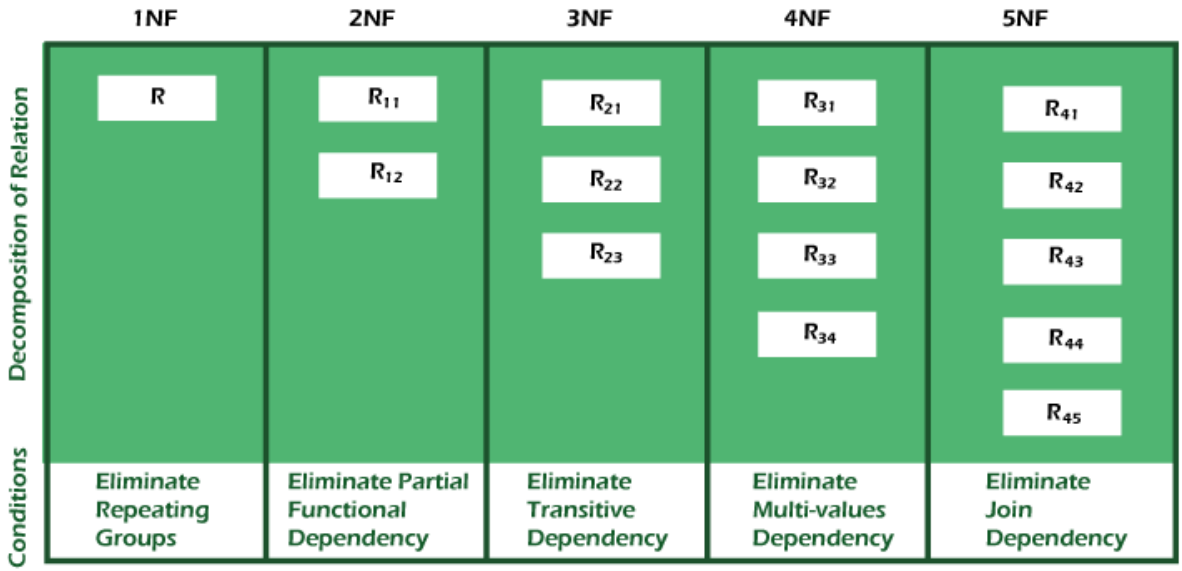
Data modification anomalies can be categorized into three types:

- **Insertion Anomaly:** Insertion Anomaly refers to when one cannot insert a new tuple into a relationship due to lack of data.
- **Deletion Anomaly:** The delete anomaly refers to the situation where the deletion of data results in the unintended loss of some other important data.
- **Updation Anomaly:** The update anomaly is when an update of a single data value requires multiple rows of data to be updated.

Types of Normal Forms:

Normalization works through a series of stages called Normal forms. The normal forms apply to individual relations. The relation is said to be in particular normal form if it satisfies constraints.

Following are the various types of Normal forms:



Normal Form	Description
1NF	A relation is in 1NF if it contains an atomic value.
2NF	A relation will be in 2NF if it is in 1NF and all non-key attributes are fully functional dependent on the primary key.
3NF	A relation will be in 3NF if it is in 2NF and no transition dependency exists.
BCNF	A stronger definition of 3NF is known as Boyce Codd's normal form.
4NF	A relation will be in 4NF if it is in Boyce Codd's normal form and has no multi-valued dependency.
5NF	A relation is in 5NF. If it is in 4NF and does not contain any join dependency, joining should be lossless.

Advantages of Normalization

- Normalization helps to minimize data redundancy.
- Greater overall database organization.
- Data consistency within the database.
- Much more flexible database design.
- Enforces the concept of relational integrity.

Disadvantages of Normalization

- You cannot start building the database before knowing what the user needs.
- The performance degrades when normalizing the relations to higher normal forms, i.e., 4NF, 5NF.
- It is very time-consuming and difficult to normalize relations of a higher degree.
- Careless decomposition may lead to a bad database design, leading to serious problems.

First Normal Form (1NF)

- A relation will be 1NF if it contains an atomic value.
- It states that an attribute of a table cannot hold multiple values. It must hold only single-valued attribute.
- First normal form disallows the multi-valued attribute, composite attribute, and their combinations.

Example: Relation EMPLOYEE is not in 1NF because of multi-valued attribute EMP_PHONE.

EMPLOYEE table:

EMP_ID	EMP_NAME	EMP_PHONE	EMP_STATE
14	John	7272826385, 9064738238	UP

20	Harry	8574783832	Bihar
12	Sam	7390372389, 8589830302	Punjab

The decomposition of the EMPLOYEE table into 1NF has been shown below:

EMP_ID	EMP_NAME	EMP_PHONE	EMP_STATE
14	John	7272826385	UP
14	John	9064738238	UP
20	Harry	8574783832	Bihar
12	Sam	7390372389	Punjab
12	Sam	8589830302	Punjab

Second Normal Form (2NF)

- In the 2NF, relational must be in 1NF.
- In the second normal form, all non-key attributes are fully functional dependent on the primary key

Example: Let's assume, a school can store the data of teachers and the subjects they teach. In a school, a teacher can teach more than one subject.

TEACHER table

TEACHER_ID	SUBJECT	TEACHER_AGE
25	Chemistry	30
25	Biology	30
47	English	35

83	Math	38
83	Computer	38

In the given table, non-prime attribute TEACHER_AGE is dependent on TEACHER_ID which is a proper subset of a candidate key. That's why it violates the rule for 2NF.

To convert the given table into 2NF, we decompose it into two tables:

TEACHER_DETAIL table:

TEACHER_ID	TEACHER_AGE
25	30
47	35
83	38

TEACHER_SUBJECT table:

TEACHER_ID	SUBJECT
25	Chemistry
25	Biology
47	English
83	Math
83	Computer

Third Normal Form (3NF)

- A relation will be in 3NF if it is in 2NF and not contain any transitive partial dependency.

- 3NF is used to reduce the data duplication. It is also used to achieve the data integrity.
- If there is no transitive dependency for non-prime attributes, then the relation must be in third normal form.

A relation is in third normal form if it holds atleast one of the following conditions for every non-trivial function dependency X → Y.

1. X is a super key.
2. Y is a prime attribute, i.e., each element of Y is part of some candidate key.

   Example:

   EMPLOYEE_DETAIL table:

EMP_ID	EMP_NAME	EMP_ZIP	EMP_STATE	EMP_CITY
222	Harry	201010	UP	Noida
333	Stephan	02228	US	Boston
444	Lan	60007	US	Chicago
555	Katharine	06389	UK	Norwich
666	John	462007	MP	Bhopal

   Super key in the table above:

1. {EMP_ID}, {EMP_ID, EMP_NAME}, {EMP_ID, EMP_NAME, EMP_ZIP}....so on

   Candidate key: {EMP_ID}

   Non-prime attributes: In the given table, all attributes except EMP_ID are non-prime.

   Here, EMP_STATE & EMP_CITY dependent on EMP_ZIP and EMP_ZIP dependent on EMP_ID. The non-prime attributes (EMP_STATE, EMP_CITY) transitively dependent on super key(EMP_ID). It violates the rule of third normal form.

That's why we need to move the EMP_CITY and EMP_STATE to the new <EMPLOYEE_ZIP> table, with EMP_ZIP as a Primary key.

EMPLOYEE table:

EMP_ID	EMP_NAME	EMP_ZIP
222	Harry	201010
333	Stephan	02228
444	Lan	60007
555	Katharine	06389
666	John	462007

EMPLOYEE_ZIP table:

EMP_ZIP	EMP_STATE	EMP_CITY
201010	UP	Noida
02228	US	Boston
60007	US	Chicago
06389	UK	Norwich
462007	MP	Bhopal

Boyce Codd normal form (BCNF)

- BCNF is the advance version of 3NF. It is stricter than 3NF.
- A table is in BCNF if every functional dependency X → Y, X is the super key of the table.
- For BCNF, the table should be in 3NF, and for every FD, LHS is super key.

Example: Let's assume there is a company where employees work in more than one department.

EMPLOYEE table:

EMP_ID	EMP_COUNTRY	EMP_DEPT	DEPT_TYPE	EMP_DEPT_NO
264	India	Designing	D394	283
264	India	Testing	D394	300
364	UK	Stores	D283	232
364	UK	Developing	D283	549

In the above table Functional dependencies are as follows:

1. EMP_ID → EMP_COUNTRY
2. EMP_DEPT → {DEPT_TYPE, EMP_DEPT_NO}

Candidate key: {EMP-ID, EMP-DEPT}

The table is not in BCNF because neither EMP_DEPT nor EMP_ID alone are keys.

To convert the given table into BCNF, we decompose it into three tables:

EMP_COUNTRY table:

EMP_ID	EMP_COUNTRY
264	India
264	India

EMP_DEPT table:

EMP_DEPT	DEPT_TYPE	EMP_DEPT_NO
Designing	D394	283
Testing	D394	300

Stores	D283	232
Developing	D283	549

EMP_DEPT_MAPPING table:

EMP_ID	EMP_DEPT
D394	283
D394	300
D283	232
D283	549

Functional dependencies:

1. EMP_ID → EMP_COUNTRY
2. EMP_DEPT → {DEPT_TYPE, EMP_DEPT_NO}

Candidate keys:

For the first table: EMP_ID
For the second table: EMP_DEPT
For the third table: {EMP_ID, EMP_DEPT}

Now, this is in BCNF because left side part of both the functional dependencies is a key.

Fourth normal form (4NF)

- A relation will be in 4NF if it is in Boyce Codd normal form and has no multi-valued dependency.
- For a dependency A → B, if for a single value of A, multiple values of B exists, then the relation will be a multi-valued dependency.

Example

STUDENT

STU_ID	COURSE	HOBBY
21	Computer	Dancing
21	Math	Singing
34	Chemistry	Dancing
74	Biology	Cricket
59	Physics	Hockey

The given STUDENT table is in 3NF, but the COURSE and HOBBY are two independent entity. Hence, there is no relationship between COURSE and HOBBY.

In the STUDENT relation, a student with STU_ID, **21** contains two courses, **Computer** and **Math** and two hobbies, **Dancing** and **Singing**. So there is a Multi-valued dependency on STU_ID, which leads to unnecessary repetition of data.

So to make the above table into 4NF, we can decompose it into two tables:

STUDENT_COURSE

STU_ID	COURSE
21	Computer
21	Math

34	Chemistry
74	Biology
59	Physics

STUDENT_HOBBY

STU_ID	HOBBY
21	Dancing
21	Singing
34	Dancing
74	Cricket
59	Hockey

QUESTIONS TO IDENTIFY NORMAL FORM

To solve the question to identify normal form, we must understand its definitions of BCNF, 3 NF, and 2NF:

Definition of 2NF: No non-prime attribute should be partially dependent on Candidate Key. i.e. there should not be a partial dependency from X → Y.

Definition of 3NF: First, it should be in 2NF and if there exists a non-trivial dependency between two sets of attributes X and Y such that X → Y (i.e. Y is not a subset of X) then

a. Either X is Super Key

b. Or Y is a prime attribute.

Definition of BCNF: First, it should be in 3NF and if there exists a non-trivial dependency between two sets of attributes X and Y such that X → Y (i.e., Y is not a subset of X) then

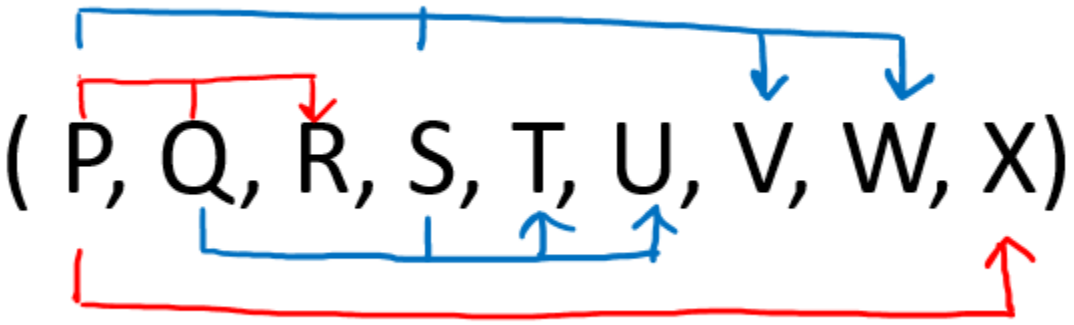
Stay

a. X is Super Key

NOTE: If a table is in BCNF then it is in 3NF, 2NF, and 1NF, similarly if the table is in 3NF Then it is in 2NF and 1NF. Hence, we can say that if a table is in the higher normal form then by default it is in lower normal form.

Question 1: Given a relation R(P, Q, R, S, T, U, V, W, X) and Functional Dependency set FD = { PQ → R, QS → TU, PS → VW, and P → X }, determine whether the given R is in which normal form?

Solution: Let us construct an arrow diagram on R using FD to calculate the candidate key.



From the above arrow diagram on R, we can see that an attribute PQS is not determined by any of the given FD, hence PQS will be the integral part of the Candidate key, i.e. no

matter what will be the candidate key, and how many will be the candidate key, but all will have PQS compulsory attribute.

Let us calculate the closure of PQS

PQS + = P Q R S T U X V W (from the closure method we studied earlier)

Since the closure of PQS contains all the attributes of R, hence **PQS is Candidate Key**

From the definition of Candidate Key (**Candidate Key is a Super Key whose no proper subset is a Super key**)

Since all key will have PQS as an integral part, and we have proved that PQS is Candidate Key, Therefore, any superset of PQS will be Super Key but not a Candidate key.

Hence there will be only **one candidate key PQS**

Since R has 9 attributes: - P, Q, R, S, T, U, V, W, X, and Candidate Key is PQS, Therefore, prime attributes (part of candidate key) are P Q and S while a non-prime attribute is R T U V W X

Given FD are { PQ → R, QS → TU, PS → VW, and P → X } and Super Key / Candidate Key is PQS

NOTE: To solve such questions, we apply reverse engineering, i.e. 1st check BCNF, if not then 3NF, if not then 2NF, and so on.

a. FD: **PQ** → **R** does not satisfy the definition of BCNF, as PQ is not Super Key, hence the table is not in BCNF (because if one dependency fails, all fails) now we check the same FD for 3NF.

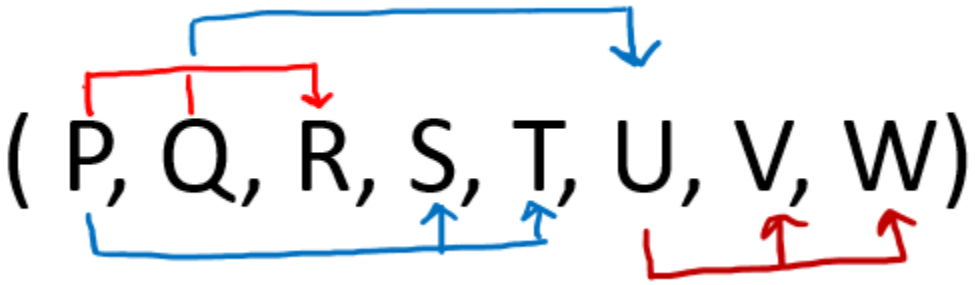
b. FD: **PQ** → **R** even does not satisfy the definition of 3NF, as PQ is not Super Key or R is not a prime attribute, hence table is not in 3NF also (because if one dependency fails, all fails) now we check same FD for 2NF

c. FD: **PQ** → **R** even does not satisfy the definition of 2NF, as PQ is not Super Key and R which is not prime attribute depending on part of the key (partial dependency), hence table is not in 2NF also (because if one dependency fails, all fails).

Hence from the above three statements, we can say that table R (P, Q, R, S, T, U, V, W, X) is in 1NF only.

Question 2: Given a relation R(P, Q, R, S, T, U, V, W) and Functional Dependency set FD = { PQ → R, P → ST, Q → U, and U → VW }, determine given R is in which normal form?

Solution: Let us construct an arrow diagram on R using FD to calculate the candidate key.



From the above arrow diagram on R, we can see that an attribute PQ is not determined by any of the given FD, hence PQ will be the integral part of the Candidate key, i.e. no matter what will be the candidate key, and how many will be the candidate key, but all will have PQ compulsory attribute.

Let us calculate the closure of PQ

PQ + = P Q R S T U V W (from the closure method we studied earlier)

Since the closure of PQ contains all the attributes of R, hence **PQ is Candidate Key**

From the definition of Candidate Key (**Candidate Key is a Super Key whose no proper subset is a Super key**)

Since all key will have PQ as an integral part, and we have proved that PQ is Candidate Key, Therefore, any superset of PQ will be Super Key but not Candidate key.

Hence there will be only **one candidate key PQ**

Since R has 8 attributes: - P, Q, R, S, T, U, V, W and Candidate Key is PQ. Therefore, prime attribute (part of candidate key) are P and Q while a non-prime attribute is R S T U V W

Given FD are { PQ → R, P → ST, Q → U, and U → VW } and Super Key / Candidate Key is PQ

NOTE: To solve such questions, we apply reverse engineering, i.e. 1st check BCNF, if not then 3NF, if not then 2NF, and so on.

a. FD: **PQ** → **R** satisfies the definition of BCNF, as PQ is Super Key, hence no need to check it for further normal forms, as it satisfies the highest one. Now we check another dependency in a reverse engineering manner.

b. FD: **P** → **ST** does not satisfy the definition of BCNF, as P is not Super Key, hence table is not in BCNF (because if one dependency fails, all fails) now we check the same FD for 3NF.

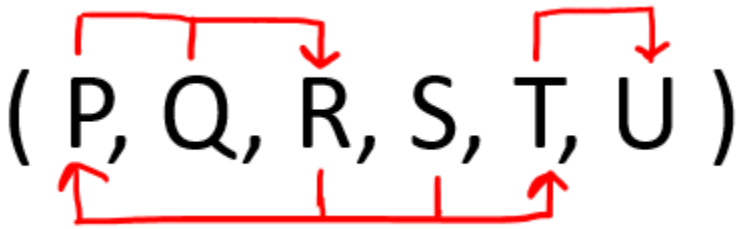
c. FD: **P** → **ST** even does not satisfy the definition of 3NF, as P is not Super Key or S T is not a prime attribute, hence table is not in 3NF also (because if one dependency fails, all fails) now we check same FD for 2NF.

d. FD: **P** → **ST** even does not satisfy the definition of 2NF, as P is not Super Key and S T which is not prime attribute depending on part of the key (partial dependency), hence table is not in 2NF also (because if one dependency fails, all fails).

Hence from the above three statements b, c, and d we can say that table R (P, Q, R, S, T, U, V, W,) is in 1NF only.

Question 3: Given a relation R(P, Q, R, S, T, U) and Functional Dependency set FD = { PQ → R, SR→ PT, T → U }, determine given R is in which normal form?

Solution: Let us construct an arrow diagram on R using FD to calculate the candidate key.



From the above arrow diagram on R, we can see that an attribute QS is not determined by any of the given FD, hence QS will be the integral part of the Candidate key, i.e. no matter what will be the candidate key, and how many will be the candidate key, but all will have QS compulsory attribute.

Let us calculate the closure of QS

QS + = QS (from the closure method we studied earlier)

Since closure QS does not contain all the attributes of R, hence QS is not the Candidate key.

On making a combination of QS with another attribute, we found that PQS and RQS determine all the attributes of R, hence **PQS and RQS are candidate keys of R.**

Since R has 6 attributes: - P, Q, R, S, T, U and Candidate Key is PQS and RQS, Therefore, prime attributes (part of candidate key) are P Q R and S while a non-prime attribute is T U

Given FD are { PQ → R, SR→ PT, T → U } and Super Key / Candidate Key is PQS and RQS

NOTE: To solve such questions, we apply reverse engineering, i.e. 1st check BCNF, if not then 3NF, if not then 2NF, and so on.

a. FD: **PQ** → **R** does not satisfy the definition of BCNF, as PQ is not Super Key, hence table is not in BCNF (because if one dependency fails, all fails) now we check the same FD for 3NF

b. FD: **PQ** → **R** satisfies the definition of 3NF, even though PQ is not Super Key but R is attributed, hence the table is in 3NF now we check other FD's using reverse engineering process.

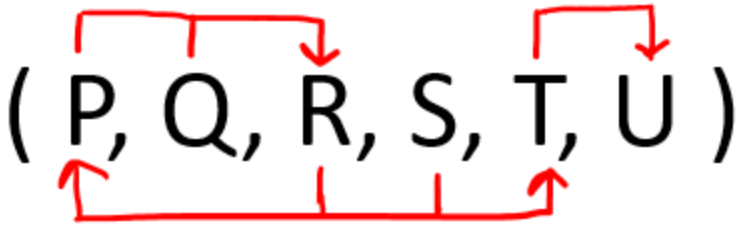
c. FD: **SR** → **PT** does not satisfy the definition of 3NF, as SR is not Super Key or P T is not prime attribute (as P is prime but the combination should be prime attribute), hence table is not in 3NF (because if one dependency fails, all fails). now we check the same FD for 2NF

d. FD: **SR** → **PT** does not satisfy the definition of 2NF, as SR is not Super Key or P T which is not prime attribute depending on part of the key (partial dependency), hence table is not in 2NF also (because if one dependency fails, all fails).

Hence from the above two statements c and d, we can say that table R (P, Q, R, S, T, U) is in 1NF only.

Question 4: Given a relation R(P, Q, R, S, T) and Functional Dependency set FD = { QR → PST, S → Q }, determine given R is in which normal form?

Solution: Let us construct an arrow diagram on R using FD to calculate the candidate key.



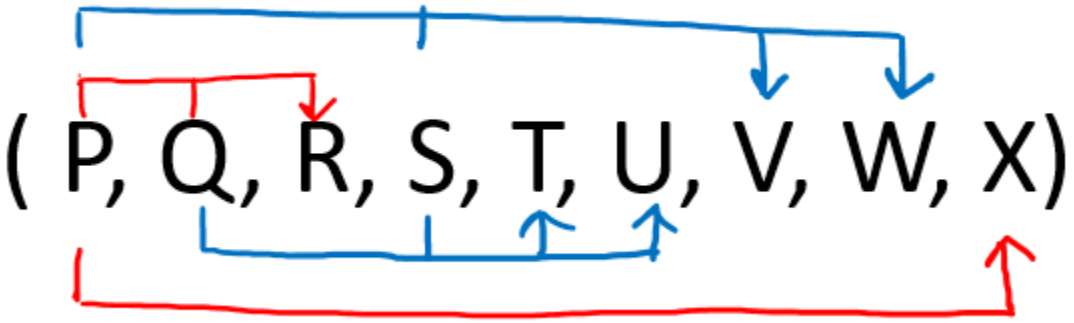
From the above arrow diagram on R, we can see that an attribute R is not determined by any of the given FD, hence R will be the integral part of the Candidate key, i.e. no matter what will be the candidate key, and how many will be the candidate key, but all will have R compulsory attribute.

Let us calculate the closure of R

R + = R (from the closure method we studied earlier)

Since closure R does not contain all the attributes of R, hence R is not Candidate key.

On making a combination of R with another attribute, we found that RS and RQ determine all the attributes of R, hence **RS and RQ are candidate keys of R.**



Since R has 5 attributes: - P, Q, R, S, T and Candidate Keys are RS and RQ, Therefore prime attributes (part of candidate key) are S Q R while a non-prime attribute is TP

Given FD are { QR → PST, S → Q } and Super Key / Candidate Key is RS and RQ

NOTE: To solve such questions, we apply reverse engineering, i.e. 1st check BCNF, if not then 3NF, if not then 2NF, and so on.

a. FD: **QR → PST** satisfies the definition of BCNF, as QR is Super Key, we check other FD for BCNF

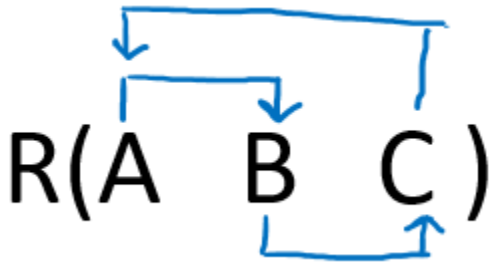
b. FD: **S → Q** does not satisfy the definition of BCNF, as S is not Super Key, hence table is not in BCNF (because if one dependency fails, all fails) now we check the same FD for 3NF

c. FD: **S → Q** satisfies the definition of 3NF, even though S is not Super Key but Q is the prime attribute, hence the table is in 3NF.

Since there were only two FD's, out of which one (QR → PST) satisfy BCNF while the other (S → Q) satisfy 3NF, hence the highest normal form is 3NF R(P, Q, R, S, T) is in 3NF.

Question 5: Given a relation R(A, B, C) and Functional Dependency set FD = { A → B, B → C, and C → A}, determine given R is in which normal form?

Solution: Let us construct an arrow diagram on R using FD to calculate the candidate key.



From the above arrow diagram on R, we can see that all the attributes are determined by all the attributes of the given FD, hence we will check all the attributes (i.e., A, B, and C) for candidate keys

Let us calculate the closure of A

A + = ABC (from the closure method we studied earlier)

Since closure A contains all the attributes of R, hence A is the Candidate key.

Let us calculate the closure of B

B + = BAC (from the closure method we studied earlier)

Since closure B contains all the attributes of R, hence B is the Candidate key.

Let us calculate the closure of C

C + = CAB (from the closure method we studied earlier)

Since closure C contains all the attributes of R, hence C is the Candidate key.

Hence three Candidate keys are: **A B and C**

Since R has 3 attributes: - A B and C, Candidate Keys are A B and C, Therefore, prime attributes (part of candidate key) are A B C while there is no non-prime attribute

Given FD are { A → B, B → C, and C → A } and Super Key / Candidate Key is A B and C

NOTE: To solve such questions, we apply reverse engineering, i.e. 1st check BCNF, if not then 3NF, if not then 2NF, and so on.

a. FD: **A → B** satisfy the definition of BCNF, as A is Super Key, we check other FD for BCNF
b. FD: **B → C** satisfy the definition of BCNF, as B is Super Key, we check other FD for BCNF
c. FD: **C à A** satisfy the definition of BCNF, as C is Super Key

Since there were only three FD's and all FD: { A → B, B → C and C → A } satisfy BCNF, hence the highest normal form is BCNF.

Therefore R(A, B, C) is in BCNF.

Conclusion: From the above three examples, we can conclude that the following steps are followed to identify the normal form for a given relational schema.

STEP 1: Calculate the Candidate Key of given R by using an arrow diagram and then using the closure of an attribute on R, such that from the calculated candidate key we can separate the prime attributes and non-prime attributes.

STEP 2: Verify each FD's in the reverse engineering process, i.e. 1st check BCNF, if not then 3NF, if not then 2NF, and so on.

STEP 3: If a table is in BCNF then it is in 3NF, 2NF, and 1NF, similarly if the table is in 3NF THEN it is in 2NF and 1NF. Hence, we can say that if a table is in the higher normal form then by default it is in lower normal form.

STEP 4: Verify first FD with Definition of BCNF (First it should be in 3NF and if there exists a non-trivial dependency between two sets of attributes X and Y such that X → Y (i.e., Y is not a subset of X) then X is Super Key.

STEP 5: If for any FD **STEP 5** fails(it signifies that table is not in BCNF), then verify that FD and remaining FD with Definition of 3NF(First it should be in 2NF and if there exist a non-trivial dependency between two sets of attributes X and Y such that X → Y (i.e. Y is not a subset of X) then X is Super Key or Y is a prime attribute

STEP 6: If for any FD STEP 6 fails (it signifies that table is not in BCNF), then verify that FD and remaining FD with Definition of 2NF (No non-prime attribute should be partially dependent on the key of table).

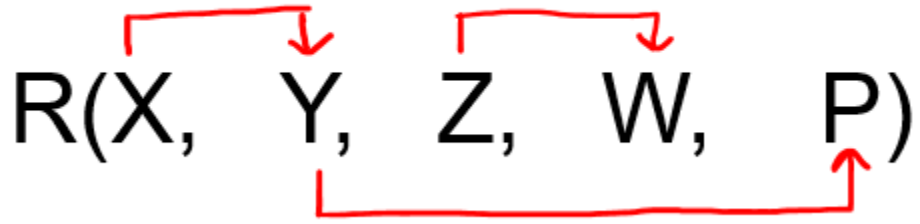
STEP 7: If for any FD STEP 7 fails (it signifies that table is not in 2NF), hence no need to check it for 1NF, as by default it is in 1NF.

STEP 8: If all the FD's satisfy the definition of BCNF then we can say that given R is in BCNF, if any FD fails for BCNF and that FD and remaining FD satisfy for 3NF then we say R is in 3NF, similarly if any FD fails for 3NF and that FD and remaining FD satisfy for 2NF then we say R is in 2NF, otherwise the table is in 1NF.

Type 2 Question

2: Given a relation R(X, Y, Z, W, P) and Functional Dependency set FD = { X → Y, Y → P, and Z → W}, determine whether the given R is in 3NF? If not convert it into 3 NF.

Solution: Let us construct an arrow diagram on R using FD to calculate the candidate key.



From above arrow diagram on R, we can see that an attributes XZ is not determined by any of the given FD, hence XZ will be the integral part of the Candidate key, i.e. no

matter what will be the candidate key, and how many will be the candidate key, but all will have XZ compulsory attribute.

Let us calculate the closure of XZ

XZ + = XZYPW (from the closure method that we studied earlier)

Since the closure of XZ contains all the attributes of R, hence **XZ is Candidate Key**

From the definition of Candidate Key (**Candidate Key is a Super Key whose no proper subset is a Super key**).

Since all key will have XZ as an integral part, and we have proved that XZ is Candidate Key, Therefore, any superset of XZ will be Super Key but not the Candidate key.

Hence there will be only **one candidate key XZ**

Definition of 3NF: First it should be in 2NF and if there exists a non-trivial dependency between two sets of attributes X and Y such that X → Y (i.e., Y is not a subset of X) then

a. Either X is Super Key
b. Or Y is a prime attribute.

Since R has 5 attributes: - X, Y, Z, W, P and Candidate Key is XZ, Therefore, prime attribute (part of candidate key) are X and Z while a non-prime attribute are Y, W, and P

Given FD are X → Y, Y → P, and Z → W and Super Key / Candidate Key is XZ

a. FD: **X** → **Y** does not satisfy the definition of 3NF, that neither X is Super Key nor Y is a prime attribute.
b. FD: **Y** → **P** does not satisfy the definition of 3NF, that neither Y is Super Key nor P is a prime attribute.
c. FD: **Z** → **W** satisfies the definition of 3NF, that neither Z is Super Key nor W is a prime attribute.

Convert the table R(X, Y, Z, W, P) into 3NF:

Since all the FD = { X → Y, Y → P, and Z → W} were not in 3NF, let us convert R in 3NF

R1(X, Y) {Using FD X → Y}

R2(Y, P) {Using FD Y → P}

R3(Z, W) {Using FD Z → W}

And create one table for Candidate Key XZ

R4(X, Z) { Using Candidate Key XZ }

All the decomposed tables R1, R2, R3, and R4 are in 2NF(as there is no partial dependency) as well as in 3NF.

Hence decomposed tables are:

R1(X, Y), R2(Y, P), R3(Z, W), and **R4(X, Z)**

Denormalization in Databases

Denormalization is a database optimization technique in which we add redundant data to one or more tables. This can help us avoid costly joins in a relational database. Note that *denormalization* does not mean 'reversing normalization' or 'not to normalize'. It is an optimization technique that is applied after normalization.
Basically, The process of taking a normalized schema and making it non-normalized is called denormalization, and designers use it to tune the performance of systems to support time-critical operations.

In a traditional normalized database, we store data in separate logical tables and attempt to minimize redundant data. We may strive to have only one copy of each piece of data in a database.

For example, in a normalized database, we might have a Courses table and a Teachers table. Each entry in Courses would store the teacherID for a Course but not the teacherName. When we need to retrieve a list of all Courses with the Teacher's name, we would do a join between these two tables.

In some ways, this is great; if a teacher changes his or her name, we only have to update the name in one place.
The drawback is that if tables are large, we may spend an unnecessarily long time doing joins on tables.
Denormalization, then, strikes a different compromise. Under denormalization, we decide that we're okay with some redundancy and some extra effort to update the database in order to get the efficiency advantages of fewer joins.

Pros of Denormalization:

1. Retrieving data is faster since we do fewer joins

2. Queries to retrieve can be simpler(and therefore less likely to have bugs), since we need to look at fewer tables.

Cons of Denormalization:

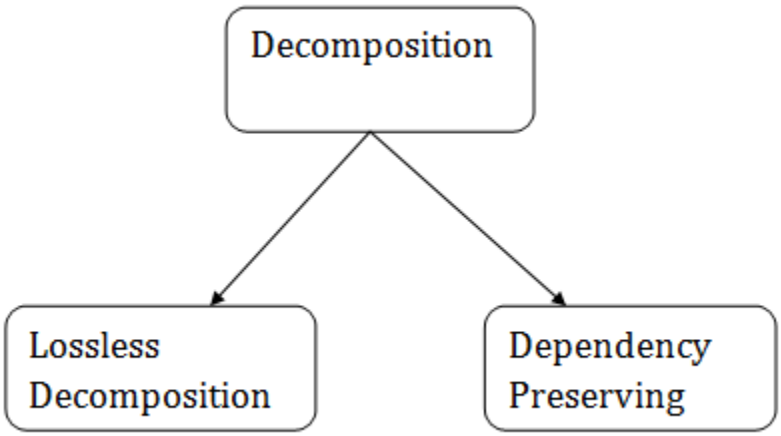
1. Updates and inserts are more expensive.
2. Denormalization can make *update* and *insert* code harder to write.
3. Data may be inconsistent.
4. Data redundancy necessitates more storage.

In a system that demands scalability, like that of any major tech company, we almost always use elements of both normalized and denormalized databases.

Relational Decomposition

- When a relation in the relational model is not in appropriate normal form then the decomposition of a relation is required.
- In a database, it breaks the table into multiple tables.
- If the relation has no proper decomposition, then it may lead to problems like loss of information.
- Decomposition is used to eliminate some of the problems of bad design like anomalies, inconsistencies, and redundancy.

Types of Decomposition



Rules for Decomposition

Whenever we decompose a relation, there are certain properties that must be satisfied to ensure no information is lost while decomposing the relations. These properties are:

1. Lossless Join Decomposition.
2. Dependency Preserving.

Lossless Join Decomposition

A lossless Join decomposition ensures two things:

- No information is lost while decomposing from the original relation.
- If we join back the sub decomposed relations, the same relation that was decomposed is obtained.

We can follow certain rules to ensure that the decomposition is a lossless join decomposition Let's say we have a relation R and we decomposed it into R1 and R2, then the rules are:

1. The union of attributes of both the sub relations R1 and R2 must contain all the attributes of original relation R.

   R1 ∪ R2 = R

2. The intersection of attributes of both the sub relations R1 and R2 must not be null, i.e., there should be some attributes that are present in both R1 and R2.

R1 ∩ R2 ≠ ∅

3. The intersection of attributes of both the sub relations R1 and R2 must be the superkey of R1 or R2, or both R1 and R2.

   R1 ∩ R2 = Super key of R1 or R2

Let's see an example of a lossless join decomposition. Suppose we have the following relation EmployeeProjectDetail as:

<EmployeeProjectDetail>

Employee_Code	Employee_Name	Employee_Email	Project_Name	Project_ID
101	John	john@demo.com	Project103	P03
101	John	john@demo.com	Project101	P01
102	Ryan	ryan@example.com	Project102	P02
103	Stephanie	stephanie@abc.com	Project102	P02

Now, we decompose this relation into EmployeeProject and ProjectDetail relations as:

<EmployeeProject>

Employee_Code	Project_ID	Employee_Name	Employee_Email
101	P03	John	john@demo.com
101	P01	John	john@demo.com
102	P04	Ryan	ryan@example.com
103	P02	Stephanie	stephanie@abc.com

The primary key of the above relation is {Employee_Code, Project_ID}.

<ProjectDetail>

Project_ID	Project_Name
P03	Project103

Project_ID	Project_Name
P01	Project101
P04	Project104
P02	Project102

The primary key of the above relation is {Project_ID}.

Now, let's see if this is a lossless join decomposition by evaluating the rules discussed above:

Let's first check the EmployeeProject ∪ ProjectDetail:

<EmployeeProject ∪ ProjectDetail>

Employee_Code	Project_ID	Employee_Name	Employee_Email	Project_Name
101	P03	John	john@demo.com	Project103
101	P01	John	john@demo.com	Project101
102	P04	Ryan	ryan@example.com	Project104
103	P02	Stephanie	stephanie@abc.com	Project102

As we can see all the attributes of EmployeeProject and ProjectDetail are in EmployeeProject ∪ ProjectDetail relation and it is the same as the original relation. So the first condition holds.

Now let's check the EmployeeProject ∩ ProjectDetail:

<EmployeeProject ∩ ProjectDetail>

Project_ID
P03
P01
P04
P02

As we can see this is not null, so the the second condition holds as well. Also the EmployeeProject ∩ ProjectDetail = Project_Id. This is the super key of the ProjectDetail relation, so the third condition holds as well.

Now, since **all three conditions hold** for our decomposition, this is a **lossless join decomposition.**

Lossless vs Lossy Decomposition

In a lossy decomposition, one or more of these conditions would fail and we will not be able to recover Complete information as present in the original relation. For example, let's say we decompose our original relation EmployeeProjectDetail into EmployeeProject and ProjectDetail relations as:

<EmployeeProject>

Employee_Code	Employee_Name	Employee_Email
101	John	john@demo.com
102	Ryan	ryan@example.com
103	Stephanie	stephanie@abc.com

The primary key of the above relation is {Employee_Code}.

<ProjectDetail>

Project_ID	Project_Name
P03	Project103
P01	Project101
P04	Project104
P02	Project102

The primary key of the above relation is {Project_ID}.

Now, the intersection EmployeeProject ∩ ProjectDetail is null. Therefore there is no way for us to map a project to its employees. Thus this is a lossy decomposition.

Dependency Preserving

- It is an important constraint of the database.
- In the dependency preservation, at least one decomposed table must satisfy every dependency.
- If a relation R is decomposed into relation R1 and R2, then the dependencies of R either must be a part of R1 or R2 or must be derivable from the combination of functional dependencies of R1 and R2.
- For example, suppose there is a relation R (A, B, C, D) with functional dependency set (A->BC). The relational R is decomposed into R1(ABC) and R2(AD) which is dependency preserving because FD A->BC is a part of relation R1(ABC).

Dependency Preserving

The second property of lossless decomposition is dependency preservation which says that after decomposing a relation R into R1 and R2, all dependencies of the original relation R must be present either in R1 or R2 or they must be derivable using the combination of functional dependencies present in R1 and R2.

Let's understand this from the same example above:

<EmployeeProjectDetail>

Employee_Code	Employee_Name	Employee_Email	Project_Name	Project_ID
101	John	john@demo.com	Project103	P03
101	John	john@demo.com	Project101	P01
102	Ryan	ryan@example.com	Project104	P04
103	Stephanie	stephanie@abc.com	Project102	P02

In this relation we have the following FDs:

- Employee_Code -> {Employee_Name, Employee_Email}
- Project_ID - > Project_Name

Now, after decomposing the relation into EmployeeProject and ProjectDetail as:

<EmployeeProject>

Employee_Code	Project_ID	Employee_Name	Employee_Email
101	P03	John	john@demo.com
101	P01	John	john@demo.com
102	P04	Ryan	ryan@example.com
103	P02	Stephanie	stephanie@abc.com

In this relation we have the following FDs:

- Employee_Code -> {Employee_Name, Employee_Email}

<ProjectDetail>

Project_ID	Project_Name
P03	Project103
P01	Project101
P04	Project104
P02	Project102

In this relation we have the following FDs:

- Project_ID - > Project_Name

As we can see that all FDs in EmployeeProjectDetail are either part of the EmployeeProject or the ProjectDetail, So this decomposition is **dependency preserving**.

Multivalued Dependency

- Multivalued dependency occurs when two attributes in a table are independent of each other but, both depend on a third attribute.
- A multivalued dependency consists of at least two attributes that are dependent on a third attribute that's why it always requires at least three attributes.

Example: Suppose there is a bike manufacturer company which produces two colors (white and black) of each model every year.

BIKE_MODEL	MANUF_YEAR	COLOR
M2011	2008	White
M2001	2008	Black
M3001	2013	White
M3001	2013	Black
M4006	2017	White
M4006	2017	Black

Here columns COLOR and MANUF_YEAR are dependent on BIKE_MODEL and independent of each other.

In this case, these two columns can be called as multivalued dependent on BIKE_MODEL. The representation of these dependencies is shown below:

1. BIKE_MODEL → → MANUF_YEAR
2. BIKE_MODEL → → COLOR

This can be read as "BIKE_MODEL multidetermined MANUF_YEAR" and "BIKE_MODEL multidetermined COLOR".

Join Dependency

- Join decomposition is a further generalization of Multivalued dependencies.
- If the join of R1 and R2 over C is equal to relation R, then we can say that a join dependency (JD) exists.
- Where R1 and R2 are the decompositions R1(A, B, C) and R2(C, D) of a given relations R (A, B, C, D).
- Alternatively, R1 and R2 are a lossless decomposition of R.
- A JD ⋈ {R1, R2,..., Rn} is said to hold over a relation R if R1, R2,....., Rn is a lossless-join decomposition.

- The *(A, B, C, D), (C, D) will be a JD of R if the join of join's attribute is equal to the relation R.
- Here, *(R1, R2, R3) is used to indicate that relation R1, R2, R3 and so on are a JD of R.

Inclusion Dependency

- Multivalued dependency and join dependency can be used to guide database design although they both are less common than functional dependencies.
- Inclusion dependencies are quite common. They typically show little influence on designing of the database.
- The inclusion dependency is a statement in which some columns of a relation are contained in other columns.
- The example of inclusion dependency is a foreign key. In one relation, the referring relation is contained in the primary key column(s) of the referenced relation.
- Suppose we have two relations R and S which was obtained by translating two entity sets such that every R entity is also an S entity.
- Inclusion dependency would be happen if projecting R on its key attributes yields a relation that is contained in the relation obtained by projecting S on its key attributes.
- In inclusion dependency, we should not split groups of attributes that participate in an inclusion dependency.
- In practice, most inclusion dependencies are key-based that is involved only keys.

Explanation: Background:

- **Lossless-Join Decomposition:**
  Decomposition of R into R1 and R2 is a lossless-join decomposition if at least one of the following functional dependencies are in F+ (Closure of functional dependencies)
- $R1 \cap R2 \rightarrow R1$

  OR
- $R1 \cap R2 \rightarrow R2$

- **Dependency Preserving Decomposition:**
  Decomposition of R into R1 and R2 is a dependency preserving decomposition if closure of functional dependencies after decomposition is same as closure of of FDs before decomposition.
  A simple way is to just check whether we can derive all the original FDs from the FDs present after decomposition.

Question :
Let R (A, B, C, D) be a relational schema with the following functional dependencies:
`A -> B, B -> C,`

`C -> D and D -> B.`

`The decomposition of R into`

`(A, B), (B, C), (B, D)`

Note that A, B, C and D are all key attributes. We can derive all attributes from every attribute.

Since Intersection of all relations is B and B derives all other attributes, relation is **lossless**.
The relation is **dependency preserving** as well as all functional dependencies are preserved directly or indirectly. Note that C -> D is also preserved with following two C -> B and B -> D.

Canonical Cover

In the case of updating the database, the responsibility of the system is to check whether the existing functional dependencies are getting violated during the process of updating. In case of a violation of functional dependencies in the new database state, the rollback of the system must take place.

A canonical cover or irreducible a set of functional dependencies FD is a simplified set of FD that has a similar closure as the original set FD.

Extraneous attributes

An attribute of an FD is said to be extraneous if we can remove it without changing the closure of the set of FD.

Example: Given a relational Schema R(A, B, C, D) and set of Function Dependency FD = { B → A, AD → BC, C → ABD }. Find the canonical cover?

rule(Armstrong Axiom).

1. B → A
2. AD → B (using decomposition inference rule on AD → BC)
3. AD → C (using decomposition inference rule on AD → BC)
4. C → A (using decomposition inference rule on C → ABD)
5. C → B (using decomposition inference rule on C → ABD)
6. C → D (using decomposition inference rule on C → ABD)

Now set of FD = { B → A, AD → B, AD → C, C → A, C → B, C → D }

The next step is to find closure of the left side of each of the given FD by including that FD and excluding that FD, if closure in both cases are same then that FD is redundant and we remove that FD from the given set, otherwise if both the closures are different then we do not exclude that FD.

Calculating closure of all FD { B → A, AD → B, AD → C, C → A, C → B, C → D }

1a. Closure B+ = BA using FD = { **B → A**, AD → B, AD → C, C → A, C → B, C → D }

1b. Closure B+ = B using FD = { AD → B, AD → C, C → A, C → B, C → D }

From 1 a and 1 b, we found that both the Closure(by including **B → A** and excluding **B → A**) are not equivalent, hence FD B → A is important and cannot be removed from the set of FD.

2 a. Closure AD+ = ADBC using FD = { B →A, **AD → B**, AD → C, C → A, C → B, C → D }

2 b. Closure AD+ = ADCB using FD = { B → A, AD → C, C → A, C → B, C → D }

From 2 a and 2 b, we found that both the Closure (by including **AD → B** and excluding **AD → B**) are equivalent, hence FD **AD → B** is not important and can be removed from the set of FD.

Hence resultant FD = { B → A, AD → C, C → A, C → B, C → D }

3 a. Closure AD+ = ADCB using FD = { B →A, **AD → C**, C → A, C → B, C → D }

3 b. Closure AD+ = AD using FD = { B → A, C → A, C → B, C → D }

From 3 a and 3 b, we found that both the Closure (by including **AD → C** and excluding **AD → C**) are not equivalent, hence FD AD → C is important and cannot be removed from the set of FD.

Hence resultant FD = { B → A, AD → C, C → A, C → B, C → D }

4 a. Closure C+ = CABD using FD = { B →A, AD → C, **C → A**, C → B, C → D }

4 b. Closure C+ = CBDA using FD = { B → A, AD → C, C → B, C → D }

From 4 a and 4 b, we found that both the Closure (by including **C → A** and excluding **C → A**) are equivalent, hence FD **C → A** is not important and can be removed from the set of FD.

Hence resultant FD = { B → A, AD → C, C → B, C → D }

5 a. Closure C+ = CBDA using FD = { B →A, AD → C, **C → B**, C → D }

5 b. Closure C+ = CD using FD = { B → A, AD → C, C → D }

From 5 a and 5 b, we found that both the Closure (by including **C → B** and excluding **C → B**) are not equivalent, hence FD **C → B** is important and cannot be removed from the set of FD.

Hence resultant FD = { B → A, AD → C, C → B, C → D }

6 a. Closure C+ = CDBA using FD = { B →A, AD → C, C → B, **C → D** }

6 b. Closure C+ = CBA using FD = { B → A, AD → C, C → B }

From 6 a and 6 b, we found that both the Closure(by including **C → D** and excluding **C → D**) are not equivalent, hence FD **C → D** is important and cannot be removed from the set of FD.

Hence resultant FD = { B → A, AD → C, C → B, C → D }

- Since FD = { B → A, AD → C, C → B, C → D } is resultant FD, now we have checked the redundancy of attribute, since the left side of FD AD → C has two attributes, let's check their importance, i.e. whether they both are important or only one.

Closure AD+ = ADCB using FD = { B →A, **AD → C**, C → B, C → D }

Closure A+ = A using FD = { B →A, **AD → C**, C → B, C → D }

Closure D+ = D using FD = { B →A, **AD → C**, C → B, C → D }

Since the closure of AD+, A+, D+ that we found are not all equivalent, hence in FD AD → C, both A and D are important attributes and cannot be removed.

Hence resultant FD = { B → A, AD → C, C → B, C → D } and we can rewrite as

FD = { B → A, AD → C, C → BD } is Canonical Cover of FD = { B → A, AD → BC, C → ABD }.